TP : Fonctionnalités avancées d'ORM (Partie 2)

Nous reprenons exactement le même contexte que la partie 1 : **Application de gestion de tâches (Task Manager)** → Utilisateurs (User) → Tâches (Task) → Nouvelles entités : Catégories (Category), Tags (Tag), Commentaires (Comment), Priorités (Priority – exemple polymorphique léger)

Objectifs de cette partie 2

Couvrir les relations les plus courantes et quelques cas avancés / utiles :

- One To One (inverse)
- One To Many + inverse (belongsTo)
- Many To Many (avec pivot)
- Has One Of Many (latestOfMany / oldestOfMany)
- Has Many Through
- Polymorphic (morphMany + morphTo)
- Polymorphic Many To Many (morphToMany)
- Eager loading + with + whenLoaded
- Contraintes sur les relations (whereHas, doesntHave, withCount, withSum, etc.)
- Accesseurs relation + scopes locaux
- touch parent on save

1 : Créer les nouveaux modèles & migrations

```
php artisan make:model Category -m -s
php artisan make:model Tag -m -s
php artisan make:model Comment -m -s -f
php artisan make:model Priority -m -s    # exemple polymorphique léger
```

Migration – categories

```
Schema::create('categories', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('color')->nullable(); // ex: #ff0000
    $table->timestamps();
});
```

Migration – tags

```
Schema::create('tags', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->timestamps();
});
```

Migration – tag_task (pivot)

```
php artisan make:migration create_tag_task_table
```

```
Schema::create('tag_task', function (Blueprint $table) {
    $table->id();
    $table->foreignId('task_id')->constrained()->cascadeOnDelete();
    $table->foreignId('tag_id')->constrained()->cascadeOnDelete();
    $table->timestamps();                // optionnel - utile pour ordered tags
par exemple
    $table->tinyInteger('priority')->default(0); // champ extra sur pivot
```

```
});
```

Migration – comments (polymorphic)

```
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->text('body');
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->morphs('commentable');      // commentable_id + commentable_type
    $table->timestamps();
});
```

Migration – priorities (exemple polymorphique – priorités sur Task ou Project futur)

```
Schema::create('priorities', function (Blueprint $table) {
    $table->id();
    $table->string('level')->default('medium'); // low / medium / high
    $table->morphs('prioritizable');
    $table->timestamps();
});
```

Migration – Tasks (Ajouter un champs pour catégorie)

```
$table->foreignId('category_id')->nullable()->constrained()->onDelete('set
null');    // ou 'cascade' / 'restrict' selon votre besoin
```

2 : Définir les relations dans les modèles

app/Models/User.php (existant)

```
public function tasks()
{
    return $this->hasMany(Task::class);
}

public function latestTask(): HasOne
{
    return $this->hasOne(Task::class)->latestOfMany();
    // ou ->oldestOfMany() pour la plus ancienne
}

public function completedTasksCount(): Attribute
{
    return Attribute::make(
        get: fn () => $this->tasks()->where('status', 'completed')->count()
    );
}
```

app/Models/Task.php

```
public function user()
{
    return $this->belongsTo(User::class);
}

public function category()
{
    return $this->belongsTo(Category::class);
}

public function tags()
{
```

```php
    return $this->belongsToMany(Tag::class)
                ->withTimestamps()
                ->withPivot('priority')
                ->orderByPivot('priority', 'desc');
}

public function comments()
{
    return $this->morphMany(Comment::class, 'commentable');
}

public function priority()
{
    return $this->morphOne(Priority::class, 'prioritizable');
}

// Exemple scope utile
public function scopeOverdue($query)
{
    return $query->where('due_date', '<', now())
                 ->where('status', '!=', 'completed');
}
```

app/Models/Category.php

```php
public function tasks()
{
    return $this->hasMany(Task::class);
}

public function activeTasks()
{
    return $this->tasks()->where('status', '!=', 'completed');
}
```

app/Models/Tag.php

```php
public function tasks()
{
    return $this->belongsToMany(Task::class)
                ->withPivot('priority');
}
```

app/Models/Comment.php

```php
public function user()
{
    return $this->belongsTo(User::class);
}

public function commentable()
{
    return $this->morphTo();
}
```

app/Models/Priority.php

```php
public function prioritizable()
{
    return $this->morphTo();
}
```

3 : Seeders pour tester les relations

Modifier ou créer database/seeders/DatabaseSeeder.php

```
public function run()
{
    User::factory(5)->create();
    $this->call([
        CategorySeeder::class,
        TagSeeder::class,
        TaskSeeder::class,
        CommentSeeder::class,
    ]);
}
```

Exemple rapide CategorySeeder :

```
$categories = ['Urgent', 'Feature', 'Bug', 'Refactoring'];
foreach ($categories as $name) {
    Category::create(['name' => $name, 'color' => fake()->hexColor()]);
}
```

TagSeeder :

```
$tags = ['frontend', 'backend', 'api', 'tests', 'security'];
foreach ($tags as $name) {
    Tag::create(['name' => $name]);
}
```

CommentFactory :

```
        return [
            'body' => $this->faker->paragraph()
        ];
```

TaskSeeder (amélioré) :

```
$user = User::first() ?? User::factory()->create();

$tasksData = [
    ['title' => 'Créer landing page', 'status' => 'in_progress', 'priority' => 3,
'due_date' => now()->addDays(5)],
    ['title' => 'Corriger bug login', 'status' => 'pending', 'priority' => 5,
'due_date' => now()->subDays(2)],
    ['title' => 'Écrire tests unitaires', 'status' => 'completed', 'priority' =>
2],
];

foreach ($tasksData as $data) {
    $task = Task::create([
        ...$data,
        'user_id'     => $user->id,
        'category_id' => Category::inRandomOrder()->first()?->id,
    ]);

    // Attacher 1-3 tags aléatoires avec priorité pivot
    $randomTags = Tag::inRandomOrder()->take(rand(1,3))->get();
    foreach ($randomTags as $tag) {
        $task->tags()->attach($tag->id, ['priority' => rand(1,10)]);
    }

    // Ajouter 1-2 commentaires
    Comment::factory()->count(rand(1,2))->create([
        'commentable_id'   => $task->id,
        'commentable_type' => Task::class,
```

```
        'user_id'          => $user->id,
    ]);

    // Ajouter une priorité polymorphique
    Priority::create([
        'level'             => collect(['low','medium','high'])->random(),
        'prioritizable_id'   => $task->id,
        'prioritizable_type' => Task::class,
    ]);
}
```

```
php artisan migrate:fresh --seed
```

4 : Tests dans Tinker (ou créer un contrôleur de démo)

```
php artisan tinker
```

```
// importer toutes les classes qu'en va utiliser
$user = User::first();

// Toutes les tâches d'un user + eager loading
$user->load('tasks.category', 'tasks.tags', 'tasks.comments.user',
'tasks.priority');
$user->tasks;

// Tâche la plus récente d'un user
$user->latestTask;

// Tâches en retard (scope)
Task::overdue()->get();

// Tâches d'une catégorie qui ont au moins un tag "security"
Category::whereHas('tasks.tags', fn($q) => $q->where('name', 'security'))->get();

// Nombre de tâches terminées par catégorie
Category::withCount(['tasks as completed_tasks_count' => fn($q) => $q-
>where('status','completed')])->get();

// Somme des priorités pivot par tag
Tag::withSum('tasks as total_priority', 'tag_task.priority')->get();

// Commentaires polymorphiques sur une tâche
$task = Task::with('comments.user')->find(1);
$task->comments;

// Priorité polymorphique
$task->priority;

// Ajouter un commentaire polymorphique
$task->comments()->create(['body' => 'Super tâche !', 'user_id' => 1]);

// Touch parent (met à jour updated_at du parent)
$task->touch();                  // met à jour task
$task->user->touch();            // met à jour user

// detach / sync tags
$task->tags()->sync([1,3,5]);    // remplace
$task->tags()->syncWithoutDetaching([2]); // ajoute sans supprimer
```

5 : Bonus – quelques patterns utiles

1. **Relation conditionnelle dans with()**

```php
Task::with(['category' => fn($q) => $q->where('color', 'like', '#f%')])->get();
```

2. **whenLoaded() dans Blade / API Resource**

```php
{{ $task->whenLoaded('category', fn() => $task->category->name, 'Sans catégorie')
}}
```

3. **Has one of many avancé**

```php
// Dernière tâche terminée par utilisateur
$user->hasOne(Task::class)->ofMany('completed_at', 'max');
```

4. **whereRelation / orWhereRelation** (depuis Laravel 8+ très utilisé)

```php
User::whereRelation('tasks', 'status', 'pending')->get();
```

Résumé – checklist relations vues

Type	Méthode	Exemple dans le TP
One To Many	hasMany	User → tasks
Inverse	belongsTo	Task → user
Many To Many	belongsToMany	Task ↔ tags (pivot priority)
Has One Of Many	hasOne + latestOfMany	User → latestTask
Morph Many	morphMany	Task → comments
Morph To	morphTo	Comment → commentable
Morph One	morphOne	Task → priority
Contraintes / agrégats	withCount, whereHas, doesntHave, withSum	Tâches par catégorie, tags

Faites tourner ces exemples dans Tinker, puis essayez de les mettre dans un petit contrôleur + vue (ou API Resource si vous préférez).